Tel Aviv University

The Iby and Aladar Fleischman Faculty of Engineering

# FPGA Laboratory

# Experiment FPGA 5 – Keyboard

Written by: Osher Stamker and Leo Segre

Based on Konstantin Berestizshevsky's work

## Lab Goals (This is also the material for the test)

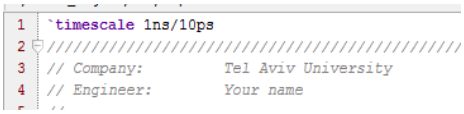
The purpose of this lab is to cover the following topics:

1. Asynchronous reset – advantage and disadvantages
2. PS2 interface - serial data transfer, transmission-frame structure, not a constantly toggling clock, how to identify an event of keypress.

## Submission Guidelines

The lab consists of 2 tasks. In each task you'll find what to submit in the preliminary report and what to submit in the final report. The preliminary report is where you will need more effort. Namely, you need to design and test all the sources of all the tasks and to submit them as a preliminary report. In the lab, you'll only need to implement the design on FPGA and deal with post-implementation debugging. Otherwise, you will not have enough time in the lab.

1. Submit a report (PDF format), as well as the XDC constraints file, and all the *.v files you have edited. All zipped together in a zip file named FPGA_5 <ID1> <ID2>
2. Each waveform must be annotated by drawing arrows that point to important signals and explaining these signals.
3. Each Verilog source/test-bench header must have both of your names in it:




4. At home you can work with the Vivado Web-Pack, which allows you to design and simulate RTL modules. Only at the lab you will be able to test your design on an actual FPGA chip.
5. It is your responsibility to either use a USB-flash drive or other cloud solutions to save your project before you leave the laboratory. The laboratory PCs are erased from time to time.

# Task 1 – PS2 interface for keyboard input

**First, create "lab5" project.** In this task, you will implement a PS2 serial interface to allow the FPGA to read input values from an external keyboard. The modules for you to implement are:

1) **Ps2_Interface** – with 3 inputs (PS2Clk, rstn, PS2Data) and 2 outputs (scancode[7:0], keyPressed) The module provides an interface with the keyboard and outputting the most recently pressed 8-bit scan-code as well as outputting a pulse "keyPressed" to notify of a new key pressing at the moment of the first make-code packet reception.
2) **Ps2_Display** – with 4 inputs (clk, rstn, keyPressed, scancode[7:0]) and 4 outputs (seg[6:0], an[3:0], dp, led). The module operates the 7-segment display and the user LED to provide visual feedback of the input inspected by the Ps2_Interface module. The functionality of this module is:
   a. **7-seg outputs (seg[6:0],an[3:0],dp).** Display the 2 hex symbols of the scan-code received by the Ps2_Interface on the two right hand side 7-segment displays (recall that there are also extended keys which transmit 4-symbols of scan-code, for them you should indicate only the last 2 symbols and to ignore the 0xE0 prefix). The 7-segment display should show the scan-code of a key until a new key is pressed, at which time it starts to show the scan-code of the new key. You should adjust the code of Seg_7_display module from the previous lab to have the most of the functionality done. To distinguish between D and 0 as well as between B and 8, you should light up the dot (dp output) when showing D and B symbols or simply use lowercase "d" and "b".
   b. **User LED output (led)**. Your module must output a strobe signal called led to indicate the event of a key pressing on the keyboard. A strobe signal is a short pulse on one of the board LEDs, yet long enough to be noticed by the naked eye.
3) **Ps2_Top** – a top level module that instantiates ps2_insterface and ps2_display and connects to the FPGA pins. Refer to Figure 1 for a high-level diagram of the design.

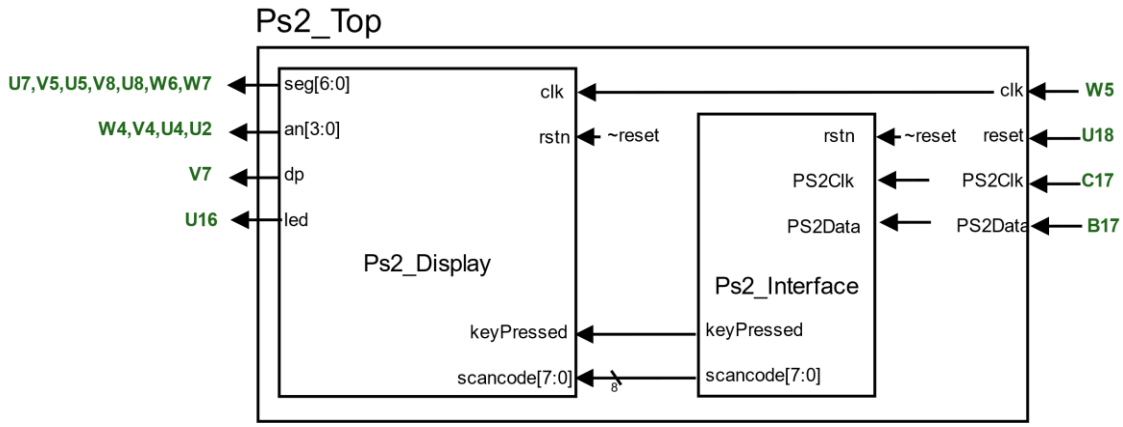



*Figure 1 - High level diagram of the design required in task 1. The green labels are the FPGA pin name whereas the black labels are the IO ports of the Verilog sources.*

## Important Design Notes:

1. There is an easy way of implementing the keyboard interface using a 22-bit shift register in your design.
2. Don't always wait to receive a 22-bit long transmission, evaluate the received bits every 11 clock cycles.
3. The keyboard-clock is not toggling all the time, but only during the transmission. Hence, waiting until receiving the whole packet will bring you to a non-toggling clock and you won't be able to perform a useful logic because the clock will be idle by this moment. Try analyzing the packet before the last cycle (before the receiving the stop bit).
4. As a part of the preliminary report you must perform a rigorous simulation to ensure a correct timing. Make sure that the "keyPressed" pulse is generated correctly – at the moment of the first pressing on the button. Make sure that despite the numerous retransmissions of the make-code, you do not generate multiple pluses. Make sure to ignore the 0xe0 packets.
5. For this design, you should use the keyboard clock as an input to your module. Disregard what the Basys3 Board Manual says.
6. Logic Analyzer Tip: Logic analyzer is unable to debug nets from different clock domains. In your design there are indeed two clock domains (one works with the slow keyboard clock and another works with the fast 100MHz system clock). Therefore, if you need to debug, first perform a debugging for the keyboard clock related nets, and when you are done with them – remove the debug and add the fast-clock related debug nets. You can also try creating two separate debug cores simultaneously on the same design.

## In the Lab:

The keyboard that we'll use is a small numeric keyboard.
Connect the keyboard to the USB port of the BASYS board,
Connect the PC to the BASYS board Jtag-port.
Program the FPGA with your implemented design and call an instructor to show him a correct functioning of your PS2 interface.

## Submit:

In the report – submit the files:
*Ps2_Interface.v, Ps2_Interface_tb.v, Ps2_Display.v, Ps2_Top.v, task1.xdc.*
Provide a screenshot of the simulation and explain its principle signals behavior by pointing with an arrow to these signals' transitions and annotating them.
Note: you need to simulate only your Ps2_Interface.v nothing more.
Explain which problems appeared during the lab and how did you overcome these problems. Re-submit the corrected files.